Trabajo Práctico Integrador - Implementación con Expresiones Regulares.
Fundamentos Teóricos de Informática.



Fechas

- **Sorteo de temas:** viernes 23 de octubre (en clase).
- **Clases de consulta:** 30/10, 6/11, 11/11 y 13/11.
- **Entrega del código y defensa:** lunes 16 de noviembre (el mismo día).
- **Recuperación (re-entrega):** miércoles 18 de noviembre.

Grupos y sorteo

- Los grupos son de **2 a 3 integrantes**. No se aceptan trabajos individuales.
- Cada grupo se anota antes del sorteo (hasta el inicio de la clase del 23/10) con los nombres de sus integrantes en la planilla que provee la cátedra
- Los temas se sortean **sin reposición**: cada grupo tiene un tema distinto y no se intercambian.

Modalidad y consigna

- El trabajo es grupal y lo desarrollan **enteramente los integrantes del grupo**. No se permite compartir código entre grupos ni copiarlo de internet (se comprueba).
- Cada grupo **investiga el formato o estándar** de su tema, lo documenta en estilo **monografía** y lo **presenta en clase** a sus compañeros.
- El programa **usa expresiones regulares** como herramienta principal. El **lenguaje es libre**, siempre que sea **interpretado** (por ejemplo Python, JavaScript, Ruby o PHP).
- El código se defiende en forma presencial: cualquier integrante tiene que poder explicar cualquier parte del programa, y lo que no se pueda explicar no se considera hecho por el grupo.
- El programa es **legible**, sigue buenas prácticas de programación y está **bien comentado**, con instrucciones de ejecución, parámetros de entrada y ejemplos.
- Las consignas describen el **funcionamiento mínimo**. Toda funcionalidad extra se tiene en cuenta al calificar.
- La falta de alguno de estos aspectos es motivo de re-entrega o desaprobación del práctico.

Qué tiene que hacer el programa (todos los temas)

Todos los temas tienen la misma estructura; lo que cambia es el contenido:

1. **Leer un texto real.** De un archivo o de una API pública. No se admiten cadenas inventadas de una en una.
2. **Extraer con ER** los datos del tema que aparecen en ese texto.
3. **Validar lo que la ER no puede validar:** dígitos verificadores, rangos, fechas que existen, totales que cierran.
4. **Transformar el resultado** en algo útil: normalizarlo, convertirlo, armar un resumen o construir un enlace.

Además:

- **Modo sin conexión:** si el programa usa una API, debe poder correr también con una muestra guardada en un archivo, para que la defensa no dependa de la red.

- **Datos personales:** si se usan datos reales (tickets, chats, documentos), se reemplazan por datos ficticios o se tachan antes de entregarlos.

Entrada de los programas (todos los temas)

- La entrada es siempre **un archivo de texto plano en UTF-8**, que se pasa como argumento al ejecutar el programa:

```
<programa> entrada.txt [opciones]
```

- Cada tema define, en su apartado **Entrada**:
  - **qué es un bloque** (un ticket, un mensaje, una receta, un partido...),
  - **qué patrón marca dónde empieza o termina cada bloque** (una línea con `---`, una línea en blanco, una línea que empieza con `#`...),
  - **qué hay dentro de cada bloque**, y
  - las **opciones** que recibe el programa.
- Los ejemplos de cada tema muestran el formato exacto. Los datos son ficticios y **algunos son inválidos a propósito**: el programa tiene que detectarlos y avisarlo, no fallar.
- Los docentes van a probar cada programa con **un archivo propio con el mismo formato**. Por eso el programa no puede depender de los datos del ejemplo.
- Si el tema usa una API, el programa guarda lo que descarga en un archivo con el formato de la entrada, y desde ahí trabaja igual que con un archivo escrito a mano.
- Las líneas vacías al final del archivo y los espacios al principio o al final de una línea no cambian el resultado.

Entregables

1. **Código fuente** con un `README` (cómo se ejecuta, parámetros, ejemplos de entrada y salida).
2. **Archivos de muestra** con los que se probó el programa, incluidos casos válidos e inválidos.
3. **Monografía** de **2 a 5 páginas** sobre el formato o estándar del tema: qué es, quién lo define, para qué sirve, cómo está compuesto, y qué expresiones regulares se usaron para reconocerlo.
4. **Presentación** para la defensa en clase.

Defensa

- Es **presencial**, el 16/11 (y el 18/11 para los grupos que no lleguen), y cada grupo tiene **10 minutos** en total.
- En ese tiempo el grupo presenta su tema, muestra el programa funcionando con el archivo de los docentes y responde preguntas sobre el código.
- Todos los integrantes tienen que poder responder sobre el trabajo.
- En la defensa, el grupo **explica una de las ER de su programa** elegida por los docentes. Los docentes podrían solicitar que se **dibuje el AFD** de una parte de ella.

Plus optativo: dónde no alcanzan las ER

Mostrar un caso de su tema que **no se pueda reconocer con una expresión regular** (anidamiento, paréntesis balanceados, cantidades que deben coincidir).

Temas

Se sortean los temas **1 a 12**.

1. Códigos de barras del supermercado

Investigar

Los estándares EAN-13, EAN-8 y UPC-A, quién los asigna (GS1) y qué indica el prefijo de país (779 para Argentina). Los códigos de **peso variable** que imprime la balanza de la fiambrería o la verdulería (los que empiezan con 2).

Fuente

Códigos tomados de productos o tickets reales. La API de Open Food Facts (sin clave) para buscar el nombre del producto a partir del código.

Entrada

`programa tickets.txt`

- **Bloque:** un ticket transcripto. Los tickets se separan con una línea que contiene solo `---`.
- **Dentro del bloque:** líneas de texto libre. Los códigos de barras aparecen dentro de las líneas, como secuencias de 8, 12 o 13 dígitos sin espacios.

```
SUPERMERCADO EJEMPLO
7790070012340 LECHE ENTERA 1L        1.250,00
2012345012509 JAMON COCIDO           1.250,00
7791234567890 YERBA 1KG              4.300,00
---
KIOSCO LA ESQUINA
77900104 CARAMELOS                     300,00
```

El programa debe

- Extraer los códigos de barras de un texto (por ejemplo, la transcripción de un ticket) y reconocer de qué tipo es cada uno.
- Verificar el **dígito de control** de cada código.
- Informar el país de origen según el prefijo.
- En los códigos de peso variable, separar el código del producto y el **peso o el precio** embebido. El grupo documenta el formato que usa un comercio concreto.

Plus: consultar Open Food Facts y mostrar el nombre del producto.

2. Tickets de compra

Investigar

Qué datos debe tener un ticket o una factura en Argentina: CUIT del comercio, tipo de comprobante (A, B, C), punto de venta y número (`00003-00001234`), CAE, IVA.

Fuente

Tickets reales del grupo, transcriptos a texto (sin datos personales).

Entrada

`programa tickets.txt`

- **Bloque:** un ticket. Los tickets se separan con una línea que contiene solo `---`.
- **Dentro del bloque**, en este orden:
  1. **Encabezado:** razón social, línea `CUIT: ...`, tipo y número de comprobante, línea `FECHA ... HORA ....`
  2. **Productos:** cada producto ocupa dos líneas. La primera tiene `cantidad x precio unitario` y la segunda la descripción y el importe.
  3. **Descuentos** (opcionales): líneas que empiezan con `DESC.` y tienen importe negativo.
  4. **Total:** una línea que empieza con `TOTAL`.

```
SUPERMERCADO EJEMPLO S.A.
CUIT: 30-71234567-1
TIQUE FACTURA B  N° 00003-00001234
FECHA 21/10/2026  HORA 18:32
2 x 850,00
GALLETITAS DE AGUA          1.700,00
1 x 1.250,00
LECHE ENTERA 1L             1.250,00
DESC. PROMO LECHE            -125,00
TOTAL                     $ 2.825,00
---
ALMACEN DON PEPE
CUIT: 30-50000001-2
TIQUE B  N° 0001-00000077
FECHA 31/09/2026  HORA 10:05
3 x 400,00
PAN FRANCES                 1.100,00
TOTAL                     $ 1.100,00
```

El programa debe

- Extraer el CUIT, el tipo y número de comprobante, la fecha y cada línea de producto (descripción, cantidad, precio unitario, importe).
- Validar el **CUIT** con su dígito verificador (módulo 11) y que la fecha exista.
- Comprobar que **cantidad × precio unitario** dé el importe de cada línea, y que la suma de líneas menos los descuentos dé el **total**.
- Mostrar un resumen: cantidad de productos, total gastado y total de descuentos.

Plus: comparar dos tickets del mismo producto en fechas distintas y calcular la variación de precio.

3. Patentes de vehículos

Investigar

Los formatos de patente argentinos: el de 1995 (`ABC 123`), el Mercosur desde 2016 (`AB 123 CD`) y los de motos (`123 ABC` y `A 123 BCD`).

Fuente

Un texto con patentes escritas de muchas formas (con y sin espacios, guiones, minúsculas), por ejemplo un listado de multas o de un estacionamiento armado por el grupo.

Entrada

`programa registros.txt`

- **Bloque:** un registro por línea.
- **Dentro de la línea:** `AAAA-MM-DD HH:MM | texto libre`. La patente aparece en cualquier lugar del texto libre, escrita de cualquier forma. Una línea puede no tener patente o tener más de una.

```
2026-10-21 08:15 | Ingresó el auto ab 123 cd al sector 2
2026-10-21 08:40 | Moto A-123-BCD estacionada en vereda
2026-10-21 09:02 | Salida de HJK123 y de la moto 123abc
2026-10-21 09:30 | Patente ilegible, se anotó ABCD 12
```

El programa debe

- Extraer las patentes del texto, sin importar cómo estén escritas.
- Normalizarlas a un formato único.
- Clasificarlas: auto o moto, formato viejo o Mercosur.
- Descartar las que no correspondan a ningún formato.

Plus: estimar el período aproximado de patentamiento a partir de las letras, con una tabla que arme el grupo investigando.

4. DNI y CUIL

Investigar

El código de barras PDF417 del DNI (qué datos contiene, separados por `@`, y qué versiones del formato existen). Cómo se forma el **CUIL** a partir del DNI: prefijo, número y dígito verificador.

Fuente

Cadenas como las que devuelve un lector al escanear el DNI. **Usar solo datos ficticios.**

Entrada

`programa documentos.txt`

- El archivo tiene **dos secciones**, cada una encabezada por una línea propia: `# ESCANEOS` y `# CUIL`.
- **Bloque en `# ESCANEOS`:** un escaneo del DNI por línea, con los campos separados por `@` (el ejemplo muestra una de las versiones del formato; las demás las documenta el grupo).
- **Bloque en `# CUIL`:** un CUIL por línea, con o sin guiones.

```
# ESCANEOS
00123456789@PEREZ@JUAN CARLOS@M@30123456@A@15/03/1985@10/06/2015@239
00987654321@GOMEZ@MARIA LAURA@F@35111222@B@29/02/1990@01/01/2018@201
# CUIL
20-30123456-3
27351112226
20-30123456-9
```

El programa debe

- Reconocer la versión del formato y extraer cada campo: apellido, nombre, sexo, número de DNI, ejemplar, fecha de nacimiento y fecha de emisión.
- Validar que las fechas existan y sean coherentes (la emisión es posterior al nacimiento).
- Calcular el **CUIL** a partir del DNI y el sexo, y validar CUILs dados con su dígito verificador.
- Mostrar los datos de forma legible y calcular la edad.

Plus: detectar cadenas adulteradas (un campo con un formato imposible).

5. Números de teléfono

Investigar

El plan de numeración argentino: características de área (de 2 a 4 dígitos), el `0` y el `15` en los celulares, y el formato internacional E.164 (`+54 9 ...`).

Fuente

Avisos clasificados, carteles o páginas de comercios con teléfonos escritos de muchas formas: `(0280) 15-412-3456`, `280 4123456`, `+54 9 280 412 3456`.

Entrada

`programa avisos.txt caracteristicas.csv`

- **`avisos.txt`** — **bloque:** un aviso. Los avisos se separan con **una línea en blanco**. Dentro del aviso hay texto libre en una o varias líneas, con cero o más teléfonos.
- **`caracteristicas.csv`:** la tabla que arma el grupo, una característica por línea: `caracteristica;localida`

```
Vendo bicicleta rodado 26, poco uso.
Llamar al (0280) 15-412-3456 o al 4421234.

Clases particulares de guitarra.
WhatsApp +54 9 2945 55-1234

Alquilo depto en Comodoro, tel 0297 446 1234 / 297-15-400-12
```

```
280;Trelew;Chubut
2945;Esquel;Chubut
297;Comodoro Rivadavia;Chubut
```

El programa debe

- Extraer todos los números de teléfono del texto.
- Distinguir fijos de celulares e identificar la localidad por la característica (con una tabla de características que arme el grupo).
- Validar que la característica y el número sumen los 10 dígitos que corresponden.
- Normalizar cada número a **E.164**.

Plus: construir el enlace de WhatsApp (`https://wa.me/549...`) de cada celular.

6. Chats de WhatsApp

Investigar

El formato del archivo que genera "Exportar chat" en Android y en iPhone (fecha, hora, autor, mensaje) y en qué se diferencian.

Fuente

Un chat exportado (con permiso de los participantes y con los nombres cambiados) o uno armado por el grupo.

Entrada

`programa chat.txt`

- El archivo es el `.txt` **tal como lo genera WhatsApp**, sin editar el formato.
- **Bloque:** un mensaje. Cada mensaje **empieza con una línea que comienza con fecha y hora**. Toda línea que no empiece así es **continuación del mensaje anterior**.
- Formato Android: `DD/MM/AA, HH:MM - Autor: texto`. Formato iPhone: `[DD/MM/AA, HH:MM:SS] Autor: texto`.
- Los avisos del sistema no tienen autor (`DD/MM/AA, HH:MM - texto`).

```
21/10/26, 18:32 - Ana: ¿Quién lleva la torta?
21/10/26, 18:33 - Bruno: Yo, pero alguien traiga velas
y también fósforos
21/10/26, 18:35 - Ana: <Multimedia omitido>
21/10/26, 18:40 - Bruno se unió usando el enlace de invitación
32/10/26, 09:00 - Ana: Se eliminó este mensaje
```

El programa debe

- Separar cada mensaje en fecha, hora, autor y texto, incluidos los mensajes de **varias líneas**.
- Reconocer los mensajes especiales: multimedia omitida, mensajes eliminados, avisos del sistema ("X se unió al grupo").
- Validar que las fechas y las horas existan.
- Mostrar estadísticas: mensajes por persona, horario de mayor actividad, día con más mensajes y enlaces compartidos.

Plus: aceptar los dos formatos (Android e iPhone) y detectar solo cuál es.

7. Recetas de cocina

Investigar

Cómo se escriben las cantidades en una receta: números, fracciones (`1 1/2`, ½), unidades (g, kg, ml, taza, cucharada, cdita) y expresiones como "una pizca" o "a gusto". Las equivalencias entre unidades.

Fuente

Recetas copiadas de sitios o libros de cocina, o la API de TheMealDB (gratuita, en inglés).

Entrada

```
programa recetas.txt --porciones 6 [--convertir ml]
```

- **Bloque:** una receta. Cada receta **empieza con una línea que comienza con `#`** seguida del nombre, y termina donde empieza la siguiente o en el fin del archivo.
- **Dentro del bloque**, en este orden:
  1. Una línea `Porciones: N`.
  2. Una línea `Ingredientes:` y debajo **un ingrediente por línea, que empieza con `-`** .
  3. Una línea `Preparación:` y debajo texto libre (el programa lo copia sin procesarlo).

```
# Panqueques
Porciones: 4
Ingredientes:
- 1 1/2 taza de harina
- 2 huevos
- 300 ml de leche
- ½ cdita de sal
- manteca a gusto
Preparación:
Mezclar todo y dejar reposar 20 minutos.

# Chocolatada
Porciones: 1
Ingredientes:
- 250 ml de leche
- 2 cdas de cacao
Preparación:
Calentar la leche y disolver el cacao.
```

El programa debe

- Extraer de cada ingrediente la cantidad, la unidad y el nombre.
- Reconocer los ingredientes sin cantidad ("sal a gusto").
- **Escalar la receta** a otra cantidad de porciones, mostrando las fracciones de forma legible (`1 1/2 taza`, no `1.5`).
- Convertir unidades cuando se pida (tazas a ml, onzas a gramos).

Plus: armar la lista de compras de varias recetas, sumando los ingredientes repetidos.

8. Horarios de comercios

Investigar

Cómo se escriben los horarios de atención y el formato `opening_hours` de OpenStreetMap (`Mo-Fr 08:00-12:00,16:00-20:00; Sa 09:00-13:00`).

Fuente

La API Overpass de OpenStreetMap (sin clave) para descargar comercios de una ciudad, por ejemplo las farmacias de Trelew. También carteles o páginas con horarios escritos en castellano ("Lunes a viernes de 8 a 12 y de 16 a 20").

Entrada

```
programa comercios.txt [--cuando "AAAA-MM-DD HH:MM"]
```

- **Bloque:** un comercio por línea.
- **Dentro de la línea:** `nombre;horario`, donde el horario está en formato `opening_hours`. Las reglas del horario se separan con `;` dentro del propio horario, así que **el nombre termina en el primer `;`**.
- Sin `--cuando`, el programa usa la fecha y hora actuales.

```
Farmacia Central;Mo-Fr 08:00-12:30,16:00-20:30; Sa 09:00-13:00
Farmacia del Puerto;24/7
Farmacia Sur;Mo-Fr 08:00-21:00; Sa,Su 10:00-14:00; PH off
Farmacia Norte;Mo-Fr 20:00-08:00
Farmacia Rara;Mo-Fr 25:00-12:00
```

El programa debe

- Interpretar los horarios en formato `opening_hours`.
- Validar que los horarios tengan sentido (horas entre 00:00 y 24:00, el cierre después de la apertura).
- Responder si un comercio **está abierto** en un día y una hora dados, y cuánto falta para que abra o cierre.
- Listar los comercios abiertos en este momento.

Plus: traducir los horarios escritos en castellano al formato `opening_hours`.

9. Direcciones y código postal

Investigar

El Código Postal Argentino (CPA, por ejemplo `U9100ABC`): qué indica cada parte y qué letra corresponde a cada provincia. Cómo se escriben las direcciones (calle, número, piso, departamento, abreviaturas como "Av." o "Bv.").

Fuente

Direcciones escritas de muchas formas (avisos, formularios, listados). La API Georef de datos.gob.ar (sin clave) para normalizar direcciones.

Entrada

`programa direcciones.txt`

- **Bloque:** una dirección por línea, escrita en texto libre.
- **Dentro de la línea:** calle y altura son obligatorias; piso, departamento, localidad, provincia y código postal pueden faltar o estar en cualquier orden después de la altura.

```
Av. Fontana 123 piso 2 dto B, Trelew, Chubut (U9100ABC)
25 de Mayo 450, CP 9100, Trelew
Bv. Oroño 1500 1° A - Rosario - Santa Fe - S2000XYZ
calle Belgrano 99, Rawson, Chubut, CPA X9103AAA
```

El programa debe

- Extraer de cada dirección la calle, la altura, el piso, el departamento, la localidad y el código postal.
- Reconocer códigos postales viejos (`9100`) y CPA (`U9100ABC`).
- Validar que la letra del CPA corresponda a la provincia indicada.
- Mostrar cada dirección normalizada en un formato único.

Plus: consultar Georef y mostrar la dirección normalizada y sus coordenadas.

10. Precios y montos de dinero

Investigar

Cómo se escriben los montos en la prensa y en los avisos (`$ 1.234,56`, `USD 300`, `u$s 300`, "2,5 millones", "$1,2 M") y los códigos de moneda ISO 4217.

Fuente

Noticias (por ejemplo los RSS de diarios) o avisos de venta. La API de dolarapi.com (sin clave) para la cotización.

Entrada

```
programa noticias.txt [--a ARS|USD] [--cotizacion 1200]
```

- **Bloque:** una noticia o aviso. Cada bloque **empieza con una línea que comienza con `##`** seguida del título, y sigue hasta el próximo `##` o el fin del archivo.
- **Dentro del bloque:** texto libre en una o varias líneas.
- `--a` indica a qué moneda convertir. La cotización se toma de `--cotizacion` o, si no se indica, de la API.

```
## Sube el precio de la nafta
El litro pasó de $ 1.150 a $1.234,56 en Trelew, un aumento del 7,4%.

## Vendo auto
Fiat Uno 2010, u$s 4.500 o $ 5,4 millones. Acepto permuta.

## Obra pública
La provincia invertirá 2,5 millones de dólares y $ 1.200 M en rutas.
```

El programa debe

- Extraer los montos del texto y reconocer la moneda.
- Resolver la ambigüedad entre punto y coma (`1.500` es mil quinientos, `1,5` es uno y medio).
- Convertir los montos expresados en palabras ("2,5 millones") a número.
- Normalizar todos los montos a un único formato y, cuando se pida, convertirlos de moneda.

Plus: extraer también los porcentajes y calcular el precio final ("$ 10.000 con 15 % de descuento").

11. Resultados de fútbol

Investigar

Cómo se informan los resultados y las fichas de un partido: resultado, goleadores con su minuto (`45+2'`), penales, goles en contra, tarjetas.

Fuente

Fichas de partidos de un torneo, tomadas de Wikipedia o de diarios deportivos.

Entrada

`programa partidos.txt`

- **Bloque:** un partido. Los partidos se separan con **una línea en blanco**.
- **Dentro del bloque**, en este orden:
  1. `Fecha N | AAAA-MM-DD`
  2. `Equipo local X - Y Equipo visitante`
  3. `Goles local:` y la lista de goles separados por coma (vacía si no hubo).
  4. `Goles visitante:` con el mismo formato.
- **Cada gol:** `Jugador minuto'`, con el minuto opcionalmente adicionado (`45+2'`) y opcionalmente `(p)` si fue de penal o `(e.c.)` si fue en contra.

```
Fecha 1 | 2026-08-16
Deportivo Trelew 2 - 1 Atlético Madryn
Goles local: Pérez 23', Gómez 90+3' (p)
Goles visitante: Ruiz 45+2'

Fecha 1 | 2026-08-16
Deportivo Rawson 0 - 0 Universitario de Trelew
Goles local:
Goles visitante:

Fecha 2 | 2026-08-23
Atlético Madryn 3 - 1 Deportivo Rawson
Goles local: Ruiz 10', Ruiz 95'
Goles visitante: Díaz 60' (e.c.)
```

El programa debe

- Extraer los equipos, el resultado y cada gol (jugador, minuto, si fue de penal o en contra).
- Validar que los minutos sean posibles y que la **cantidad de goles listados coincida con el resultado**.
- Calcular la **tabla de posiciones** del torneo (puntos, partidos jugados, goles a favor y en contra, diferencia).
- Mostrar la tabla de goleadores.

Plus: contemplar los partidos definidos por penales.

12. Cancioneros con acordes

Investigar

La notación de los acordes (cifrado americano `A-G`, alteraciones `#` y `b`, tipos como `m`, `7`, `maj7`, `sus4`, `dim`, bajo `/B`) y su equivalencia con la notación latina (Do, Re, Mi).

Fuente

Cancioneros con acordes sobre la letra, de canciones de dominio público (por ejemplo, folklore tradicional) o escritas por el grupo.

Entrada

`programa cancion.txt --semitonos N [--latino]`

- **Bloque:** una canción. Cada canción **empieza con una línea que comienza con `#`** seguida del título; puede seguir una línea opcional `Tono: X`.
- **Dentro del bloque:** líneas de acordes y líneas de letra. Una **línea de acordes** es la que solo contiene acordes separados por espacios; los espacios importan, porque ubican cada acorde sobre la sílaba. Toda otra línea es letra. Una línea en blanco separa estrofas.
- `N` puede ser positivo o negativo (`--semitonos -2`).

```
# Canción de ejemplo
Tono: G
G          D/F#      Em
Camino por la costa al sol
C            D7        G
miro el mar y pienso en vos

Am7      Bm     Cmaj7
vuelve la marea otra vez
Dsus4   D      G     Hx7
y se lleva todo lo que sé
```

El programa debe

- Distinguir las líneas de acordes de las líneas de letra.
- Extraer cada acorde y separarlo en nota, alteración, tipo y bajo.
- Rechazar acordes mal escritos.
- **Transponer** la canción un número de semitonos, manteniendo la alineación con la letra.

Plus: convertir entre cifrado americano y latino.

Reserva A. Indicaciones de medicamentos

Investigar

Cómo se escriben las indicaciones de una receta médica o un prospecto: dosis, unidades (mg, ml, gotas, comprimidos), frecuencia ("cada 8 h", "c/12hs") y duración ("por 7 días").

Fuente

Indicaciones transcriptas de recetas o prospectos (sin datos personales).

Entrada

`programa indicaciones.txt --inicio "AAAA-MM-DD HH:MM"`

- **Bloque:** una indicación por línea.
- **Dentro de la línea:** `Medicamento: indicación en texto libre`. El nombre del medicamento termina en el primer `:`.

```
Amoxicilina 500 mg: 1 comprimido cada 8 h durante 7 días
Ibuprofeno 400: 1 comp. c/12hs por 3 dias
Jarabe para la tos: 10 ml cada 6 horas durante 5 días
Gotas oftálmicas: 2 gotas c/ 30 hs por 2 semanas
```

El programa debe

- Extraer de cada indicación el medicamento, la dosis, la frecuencia y la duración.
- Validar que la frecuencia y la duración sean coherentes (cada 8 h son 3 tomas por día).
- Generar la **agenda de tomas** a partir de una fecha y una hora de inicio.

Plus: exportar la agenda a un archivo `.ics` para importarla en un calendario.

Reserva B. Partidas de ajedrez

Investigar

El formato PGN y la notación algebraica (`Nf3`, `exd5`, `O-O`, `e8=Q+`), también en castellano (`Cf3`, `A`, `T`, `D`, `R`).

Fuente

Partidas descargadas de la API de Lichess (sin clave) o de bases públicas.

Entrada

`programa partidas.pgn [--castellano]`

- El archivo está en **formato PGN estándar**, como lo descarga Lichess.
- **Bloque:** una partida. Cada partida **empieza con líneas de encabezado `[Etiqueta "valor"]`**, sigue una línea en blanco y luego las jugadas, que pueden ocupar varias líneas y **terminan con el resultado** (`1-0`, `0-1`, `1/2-1/2` o `*`).

```
[Event "Torneo del club"]
[White "Ana"]
[Black "Bruno"]
[Result "1-0"]

1. e4 e5 2. Bc4 Nc6 3. Qh5 Nf6 4. Qxf7# 1-0

[Event "Torneo del club"]
[White "Carla"]
[Black "Diego"]
[Result "1/2-1/2"]

1. d4 d5 2. c4 e6 4. Nc3 {la numeración salta} Nf6 0-1
```

El programa debe

- Leer los encabezados de la partida (`[White "..."]`, `[Result "..."]`).
- Extraer cada jugada y clasificarla: captura, enroque, coronación, jaque, mate.
- Validar que la numeración de las jugadas sea correlativa y que el resultado del encabezado coincida con el final.
- Traducir la partida entre notación inglesa y castellana.

Plus: los comentarios `{...}` y las variantes `(...)` pueden anidarse; es el caso natural de "dónde no alcanzan las ER".